Week 2: Double Buffering Using Multi-Threading

Learning Objectives

By the end of this lab, students will be able to:

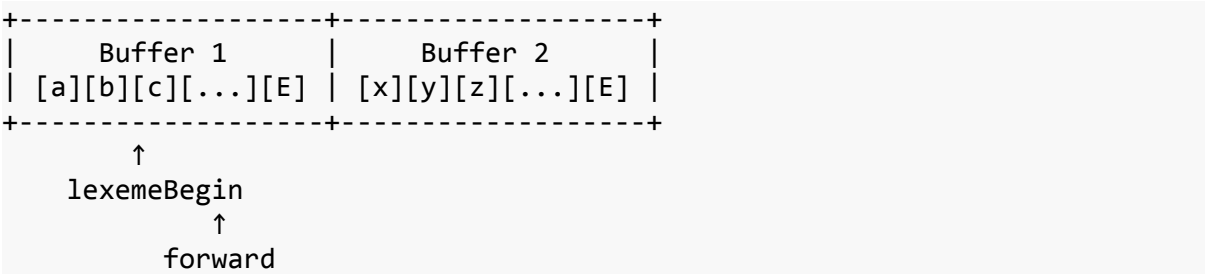
- Understand the concept of double buffering in lexical analysis
- Implement multi-threaded I/O for efficient file reading
- Manage buffer transitions seamlessly
- Handle end-of-file conditions properly

Theoretical Background

Reference: Dragon Book Section 3.2 (Input Buffering)

In lexical analysis, reading input character-by-character is inefficient. Double buffering uses two alternating buffers to minimize I/O operations. While one buffer is being processed, the other can be refilled asynchronously.

Double Buffering Concept



- **lexemeBegin**: Points to the start of the current lexeme
- **forward**: Scans ahead to find the end of the lexeme
- **E**: Sentinel character (EOF marker)

Multi-Threading Benefit

Using separate threads for:

1. **Reader Thread**: Reads fr

   \\
   om file into inactive buffer
2. **Scanner Thread**: Processes active buffer for tokens

This overlap of I/O and processing improves performance significantly.

Lab Tasks

Task 1: Buffer Management

Implement a BufferManager class with:

- Two buffers of configurable size (e.g., 4096 bytes)
- Sentinel markers at buffer ends
- Seamless transition between buffers

Task 2: Multi-Threaded Reader

Implement a producer-consumer pattern:

- **Producer (Reader Thread)**: Reads file chunks into buffers
- **Consumer (Scanner Thread)**: Processes characters from buffers
- **Synchronization**: Use semaphores or condition variables

Task 3: Character Stream Interface

Provide a clean interface for the lexical analyzer:

```
char getNextChar()      // Returns next character
void ungetChar()        // Puts back one character
string getLexeme()      // Returns current lexeme
void resetLexemeBegin() // Marks new lexeme start
```

Input Specification

Input: A large source code file (minimum 100KB recommended for testing performance)

Sample Input File (test_program.txt):

```
int main() {
    int counter = 0;
    float result = 3.14159;

    while (counter < 1000000) {
        result = result + 0.001;
        counter = counter + 1;
    }

    return 0;
}
// Repeat similar code blocks to create a large file
```

Output Specification

Output:

1. Performance metrics comparing single-buffer vs double-buffer
2. Character-by-character output demonstrating buffer transitions
3. Statistics on buffer switches

Sample Output:

```
Buffer Configuration:
- Buffer Size: 4096 bytes
- Total File Size: 150000 bytes
```

```
Reading Statistics:
- Total buffer switches: 36
- Average fill time: 0.5ms
- Processing time (single buffer): 450ms
- Processing time (double buffer): 280ms
- Performance improvement: 37.8%

Buffer Transition Demo:
[Buffer 1 active] Position 4090: 'i' 'n' 't' ' ' 'm'
[Buffer switch at position 4095]
[Buffer 2 active] Position 0: 'a' 'i' 'n' '(' ')' ' '
...

Characters processed: 150000
EOF reached successfully.
```

Deliverables

1. Source code with multi-threaded buffer implementation
2. Performance comparison report (single vs double buffer)
3. Test results with files of varying sizes

Evaluation Criteria

Criteria	Marks
Correct buffer management	25%
Thread synchronization	25%
Seamless buffer transitions	20%
Performance improvement demonstrated	20%
Code quality and documentation	10%